

LECTURE 23

Contiguous Memory Allocation and Segmentation

CSC323 · Operating Systems

COMSATS University Islamabad, Lahore Campus

In This Lecture

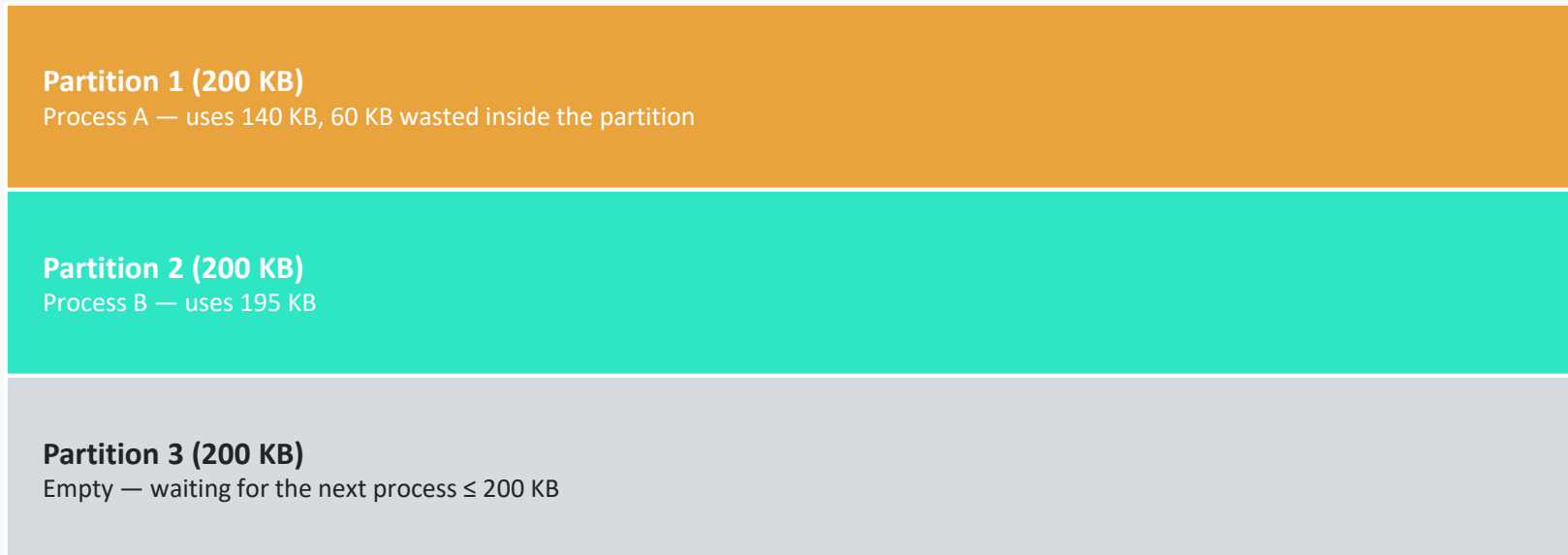
- Fixed partitioning (MFT) vs. dynamic partitioning (MVT) — two ways to divide memory among processes
- The three classic allocation strategies for dynamic partitioning — First-Fit, Best-Fit, Worst-Fit — worked through one shared example
- Internal vs. external fragmentation, and compaction as external fragmentation's cure
- Segmentation: a program as a collection of logically distinct, independently sized segments
- The segment table and the protection bits that make segmentation hardware enforceable

Contiguous Memory Allocation

- Under contiguous allocation, each process occupies a single, continuous range of physical addresses — exactly the base/limit model from Lecture 22
- Two historical approaches differ in whether partitions are fixed in advance or carved out on demand:
- **Fixed partitioning (MFT)** — partitions decided once, at boot time
- **Dynamic partitioning (MVT)** — partitions sized exactly to each process, carved from a pool of free memory as processes arrive

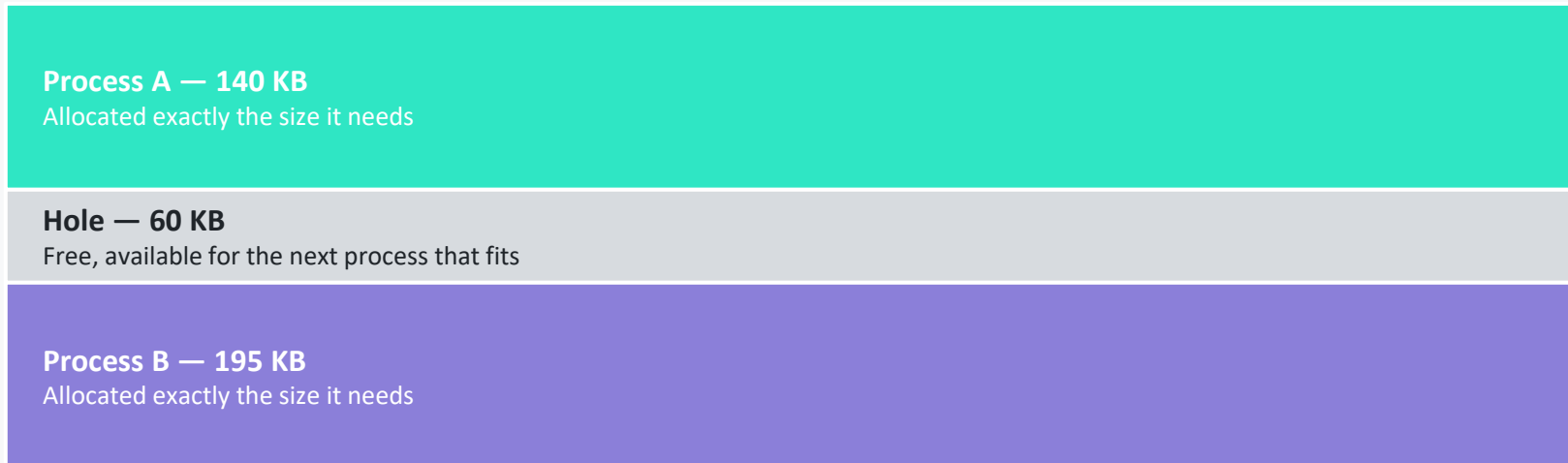
Fixed Partitioning (MFT)

- MFT — Multiprogramming with a Fixed number of Tasks — divides memory into a fixed set of partitions once, at boot time, one process per partition
- Simple to implement — but a process smaller than its partition wastes the leftover space, and a process larger than every partition cannot run at all



Dynamic Partitioning (MVT)

- MVT — Multiprogramming with a Variable number of Tasks — sizes each partition exactly to the process loading into it, carved from a pool of free holes
- No space wasted inside a partition — but free memory fragments into a scattered set of holes of different sizes as processes come and go



Allocation Strategies for Dynamic Partitioning

- **First-Fit** — scan the hole list in address order, allocate from the first hole big enough. Fast — stops at the first fit.
- **Best-Fit** — search the entire list, allocate from the smallest hole still big enough. Minimizes waste per allocation, but leaves tiny unusable slivers; slower to search.
- **Worst-Fit** — allocate from the largest hole available. Leftover fragment more likely to stay a useful size, but burns through large holes quickly.

Worked Example: Starting Holes and Requests

Starting free-hole list (address order):

Hole	H1	H2	H3	H4	H5
Size	150 KB	350 KB	90 KB	500 KB	220 KB

Requests arrive in this order:



Each strategy applies this same sequence of requests to the same starting hole list — watch where their choices diverge.

First-Fit vs. Best-Fit vs. Worst-Fit

First-Fit

Req	From	Leftover
R1=130	H1(150)	20 KB
R2=300	H2(350)	50 KB
R3=200	H4(500)	300 KB

Best-Fit

Req	From	Leftover
R1=130	H1(150)	20 KB
R2=300	H2(350)	50 KB
R3=200	H5(220)	20 KB

Worst-Fit

Req	From	Leftover
R1=130	H4(500)	370 KB
R2=300	H4(370)	70 KB
R3=200	H2(350)	150 KB

- First-Fit: R1→H1 (leftover 20), R2→H2 (leftover 50), R3→H4 (leftover 300)
- Best-Fit: R1→H1 (leftover 20), R2→H2 (leftover 50), R3→H5 (leftover 20)
- Worst-Fit: R1→H4 (leftover 370), R2→H4 again (leftover 70), R3→H2 (leftover 150)

No strategy is universally best — First-Fit is usually fastest; Best-Fit minimizes per-allocation waste but accumulates tiny slivers; Worst-Fit keeps leftovers large but exhausts big holes quickly.

Where Each Strategy Allocated R3 = 200 KB

First-Fit → H4

First hole in scan order big enough — 500 KB

Best-Fit → H5

Smallest adequate hole — 220 KB, tightest leftover (20 KB)

Worst-Fit → H2

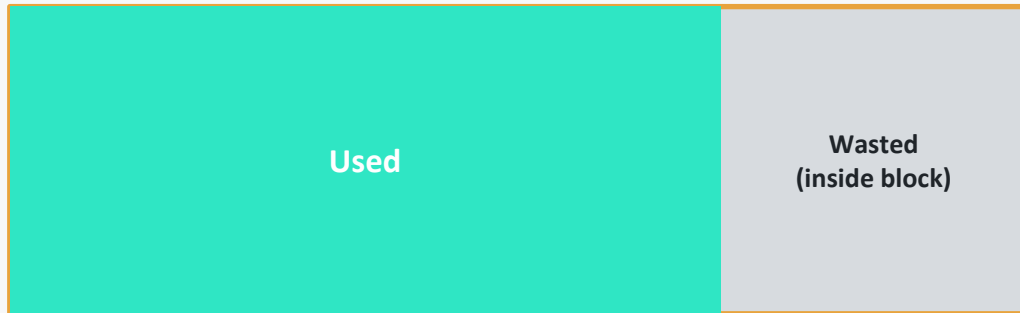
Largest hole remaining — 350 KB, largest leftover (150 KB)

- For R1 and R2, First-Fit and Best-Fit happened to agree — the smallest adequate hole was, by coincidence, also the first one First-Fit's scan reached
- Worst-Fit diverged from both immediately, by design, since it deliberately avoids the smallest adequate hole every time
- By R3, every strategy's own history of earlier allocations reshaped its hole list differently enough that all three disagree

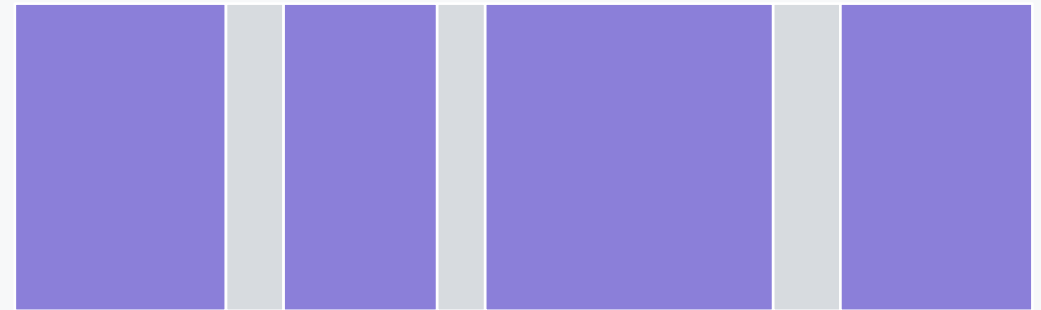
Internal vs. External Fragmentation

- **Internal fragmentation** — allocated memory is larger than requested; the extra space sits wasted inside the partition. Inherent to fixed partitioning (MFT).
- **External fragmentation** — enough total free memory exists, but scattered across holes each individually too small. Inherent to dynamic partitioning (MVT).

Internal fragmentation



External fragmentation



Scattered unusable gaps between allocated blocks — no single gap is big enough for a new request

Compaction

- Compaction addresses external fragmentation directly: shuffle every allocated block toward one end of memory, consolidating scattered holes into one large block
- It is expensive — every byte of every relocated process must be physically copied
- Only possible if every process can be relocated freely while running — requiring execution-time (dynamic) binding and relocation-register-style MMU support
- Under compile-time or load-time binding, a process's addresses are already fixed and cannot simply be moved

Non-Contiguous Allocation: Segmentation

- Segmentation views a program as a collection of logically distinct segments — code, stack, heap, and so on — each sized naturally for what it actually contains
- A logical address under segmentation is a pair: (segment number, offset within that segment) — not a single flat number
- This matches how a programmer already organizes a program conceptually, rather than forcing everything into one flat address space

The Segment Table

Segment	Base	Limit
0 — Code	4300	1200
1 — Heap	6700	2500
2 — Stack	9400	1000
3 — Shared library	12100	1800

logical address (2, 150) → 9400 + 150 = 9550

"150 bytes into the Stack segment" is translated by adding the offset to that segment's base — exactly as long as 150 does not exceed segment 2's limit of 1000.

Segmentation Hardware and Protection

- The hardware is, conceptually, a small array of (base, limit) pairs — one per segment — forming the segment table itself
- Plus a set of protection bits attached to each individual segment, not to the process as a whole
- **Code segment** — typically marked read/execute-only; the hardware traps an attempt to write to it
- **Stack or Heap segment** — marked read/write but not executable, stopping classic buffer-overflow attacks from running injected code off the stack

Finer-grained than base/limit alone — A single base/limit pair protects a process from every other process, but treats everything inside it identically. Segmentation's per-segment bits add a second layer: protecting a process even from itself.

Key Takeaways

- Fixed partitioning (MFT) divides memory into fixed-size partitions set once at boot — simple, but wastes memory via internal fragmentation
- Dynamic partitioning (MVT) sizes each partition exactly to the process — eliminates internal fragmentation, but introduces external fragmentation
- First-Fit (fast), Best-Fit (minimizes per-allocation waste, leaves slivers), and Worst-Fit (keeps leftovers large, burns big holes fast) can pick different holes for the same requests
- Compaction consolidates scattered holes into one block, but is expensive and only possible under execution-time binding
- Segmentation organizes a program as logically distinct, independently sized segments; a segment table (base, limit, protection bits) makes this enforceable